

تمام يا باشا 🔥 أنا معاك للآخر في المشروع ده. ابعت أي ملف، وأنا هشرجهولك كأننا قاعدين جنب بعض بنبنيه من الصفر.

طريقة الشرح هتبقى كده:

شرح كل سطر حرفيًا (1)

يعني لو عندك مثلاً

```
JavaScriptconstmongoose=require("mongoose");
```

هشركلك:

- ليه استخدمناها؟ const
 - يعني إيه متغير ثابت؟
 - require؟ يعني إيه
 - package؟ بيحيب الـ Node.js إزاي
 - أصلاً؟ mongoose إيه هو
 - مباشرة؟ MongoDB ليه بنستخدمه بدل ما نتعامل مع
-

شرح المنطق وراء الكود (2)

مش بس "السطر ده بيعمل كذا"، لا هقولك

ليه المبرمج كتب السطر ده هنا؟

مثلاً:

```
JavaScriptmongoose.connect(process.env.DB_URL);
```

هنفهم:

- ليه الاتصال بالداتايز بيكون في البداية؟
- process.env؟ ليه استخدمنا
- Environment Variables؟ يعني إيه
- مباشرة؟ URL ليه مش بنكتب الـ

- يشتغل؟ app إليه اللي بيحصل ورا الكواليس لما ال
-

3) Promises وال Async / Await شرح ال

لو لقيت:

```
JavaScriptasyncfunctionconnectDB() {  
  awaitmongoose.connect(url);  
}
```

هنفكها حنة حنة:

- async؟ يعني إليه
 - Promise؟ فيه الفانكشن رجعت
 - أصلاً؟ Promise يعني إليه
 - then/catch و async/await؟ فيه الفرق بين
 - Node.js بتاع Event Loop إليه اللي بيحصل في ال
-

4) Node.js فهم بنية مشروع

هنربط الملفات ببعض:

مثلاً:

```
project/  
|  
├── server.js  
├── app.js  
├── config/  
│   └── db.js  
├── models/  
│   └── userModel.js  
├── controllers/  
│   └── userController.js  
├── routes/  
│   └── userRoutes.js  
└── middleware/
```

هتعرف:

- مين بيستدعي مين؟
 - response? بيمشي من أول ما ييجي لحد ما يرجع request إزاي الـ
 - إيه؟ folder دور كل
 - ليه اتقسم بالشكل ده؟
-

5) بالتفصيل MongoDB و Mongoose شرح

زي Model لو عندك

```
JavaScriptconst userSchema = new mongoose.Schema({  
  name: String,  
  age: Number  
});
```

هنفهم:

- Schema يعني إيه
 - Document و Collection و Model و Schema الفرق بين
 - بتخزن البيانات داخلياً MongoDB إزاي
 - new Schema ليه عملنا
 - هنا object ليه استخدمنا
-

6) هسألك أسئلة صغيرة أثناء الشرح

عشان أتأكد إنك فهمت

مممكن أسألك Database مثلاً بعد ملف الـ

إيه اللي ممكن يحصل؟ `mongoose.connect()` من `await` لو شيلنا

أو:

مش في الكود؟ `.env` ليه بنحط بيانات الاتصال في

لو الكود فيه طريقة احترافية أو حاجة قديمة هقولك 7)

نزي:

- Best Practice دي 👍
 - دي شغالة بس مش أفضل طريقة ⚠️
 - Bug دي ممكن تسبب 🚨
-

في الآخر هتقدر تكتب الملف بنفسك 8)

بعد ما نخلص أي ملف هعملك ملخص بالشكل ده:

وظيفته db.js ملف:

- 1- استيراد mongoose
- 2- إنشاء اتصال بقاعدة البيانات
- 3- التعامل مع أخطاء الاتصال
- 4- الاتصال لباقي المشروع function تصدير

وبعدين هقولك:

جرب دلوقتي تكتبه من ذاكرتك، ولو نسيت حاجة ارجع للشرح.

👉 ابعت أول ملف بقى

عشان وأنا بشرح أربطه بمكانه في (config/db.js مثلاً) ويفضل تبعت **اسم الملف فوق الكود** المشروع.

ولو الملف كبير ابعت على أجزاء، وأنا هتايح معاك جزء جزء لحد ما يبقى فاهمين المشروع كله.

```
models/cart-item  const Sequelize=require('sequelize')  const
sequelize=require('../util/database')  const CartItem=sequelize.define('cartItem',
  { id:{ type:Sequelize.INTEGER, autoIncrement:true, allowNull:false,
primaryKey:true }, quantity:Sequelize.INTEGER  })  module.exports=CartItem
=====  models/cart  const Sequelize=require('sequelize')  const
sequelize=require('../util/database')  const Cart=sequelize.define('cart',{ id:
  { type:Sequelize.INTEGER, autoIncrement:true, allowNull:false,
primaryKey:true }  })  module.exports=Cart  =====  models/oreder-item
const Sequelize=require('sequelize')  const sequelize=require('../util/database')
```

```

const OrderItem=sequelize.define('orderItem',{ id:{ type:Sequelize.INTEGER,
autoIncrement:true, allowNull:false, primaryKey:true },
quantity:Sequelize.INTEGER }) module.exports=OrderItem =====
model/order const Sequelize=require('sequelize') const
sequelize=require('../util/database') const Order=sequelize.define('order',{ id:
{ type:Sequelize.INTEGER, autoIncrement:true, allowNull:false,
primaryKey:true } }) module.exports=Order ===== model/product
const mongoose=require('mongoose') const getDb=require('../util/database').getDb
class Product{ constructor(title,price,description,imageUrl,id,userId)
{ this.title=title this.price=price this.description=description
this.imageUrl=imageUrl if(id){ this._id=new mongoose.ObjectId(id) }
this.userId=userId } save(){ const db=getDb() let dbOp if(this._id)
{ dbOp=db.collection('products').updateOne({_id:this._id},{ $set:this }) }
else{ dbOp=db.collection('products').insertOne(this) } return
dbOp .then(result=>{ console.log(result) }).catch(err=>{ console.log(err) }) }
static fetchAll(){ const db=getDb() return
db.collection('products').find().toArray() .then(products=>{ console.log(products
) return products }) .catch(err=>{ console.log(err) }) } static
findById(prodId){ const db=getDb() return
db.collection('products').find({_id:new
mongoose.ObjectId(prodId)}).next() .then(product=>{ console.log(product)
return product }) .catch(err=>{ console.log(err) }) } static
deleteById(prodId){ const db=getDb() return
db.collection('products').deleteOne({_id:new
mongoose.ObjectId(prodId)}) .then(result=>{ console.log('deleted
Successfully') }) .catch(err=>{ console.log(err) }) } }
module.exports=Product اشمعا كتبت هنا كل الفانكشن دي ابقي اشرحها مع الباقي
===== model/user const mongoose = require("mongoose"); const
getDb = require("../util/database").getDb; const ObjectId = mongoose.ObjectId;
class User { constructor(userName, email, cart, id) { this.userName = userName;
this.email = email; this.cart = cart; this._id = id; } save() { const db = getDb();
return db .collection("users") .insertOne(this) .then((result) =>
{ console.log(result); }) .catch((err) => { console.log(err); }); }
addToCart(product) { const cartProductIndex = this.cart.items.findIndex((cp) => {
return cp.productId.toString() === product._id.toString(); }); let newQuantity =
1; const updatedCartItems = [...this.cart.items]; if (cartProductIndex >= 0)
{ newQuantity = this.cart.items[cartProductIndex].quantity + 1;
updatedCartItems[cartProductIndex].quantity = newQuantity; } else
{ updatedCartItems.push({ productId: new ObjectId(product._id), quantity:

```

```

newQuantity, }); } const updatedCart = { items: updatedCartItems, }; const
    db = getDb(); return db .collection("users") .updateOne( { _id: new
    ObjectId(this._id) }, { $set: { cart: updatedCart } }, ); } getCart() { const db =
    getDb(); const productIds = this.cart.items.map((i) => { return i.productId; });
    return db .collection("products") .find({ _id: { $in:
    productIds } }) .toArray() .then(products=>{ return
    products.map( p=>{ return { ...p, quantity: this.cart.items.find(i=>{ return
    i.productId.toString() === p._id.toString() }).quantity } } ) }); }
    deleteItemFromCart(productId){ const
updatedCartItems=this.cart.items.filter(item=>{ if (!item?.productId) return false
return item.productId.toString() !== productId.toString() }) const db = getDb();
return db .collection("users") .updateOne( { _id: new ObjectId(this._id) },
{ $set: { cart: {items:updatedCartItems} } }, ); } addOrder(){ const db=getDb()
return this.getCart().then(products=>{ const order={ items:products, user:
    { _id:new ObjectId(this._id), name:this.name } } return
db.collection('orders').insertOne(order) }) .then(result=>{ this.cart={items:[]}
return db .collection("users") .updateOne( { _id: new ObjectId(this._id) },
    { $set: { cart: {items:[]} } }, ); }) } getOrders(){ const db=getDb() return
db.collection('orders').find({'user._id':new ObjectId(this._id)}).toArray() } static
findById(userId) { const db = getDb(); return db .collection("users") .findOne({
_id: new ObjectId(userId) }) .then((user) => { console.log(user); return user; })
    .catch((err) => { console.log(err); }); } } module.exports = User;
===== util/database const dns = require('dns');
dns.setServers([ '8.8.8.8', '8.8.4.4' ]); const { MongoClient } =
    require('mongodb'); const uri =
'mongodb+srv://blalsmyr512_db_user:XhEm6lmXxpQsphJT@cluster0.g3no2bx.mo
ngodb.net/?appName=Cluster0'; let _db const mongoConnect = callback =>
    { MongoClient.connect(uri) .then(client => { console.log('MongoDB
    Connected'); _db=client.db() callback(); }) .catch(err => { console.error(err);
    throw err }); }; const getDb()=>{ if(_db){ return _db } throw 'no database
    found' } exports.mongoConnect=mongoConnect exports.getDb=getDb
===== app.js const path = require("path"); const express =
    require("express"); const bodyParser = require("body-parser"); const
    errorController = require("./controllers/error"); const
mongoConnect=require('./util/database').mongoConnect const app = express();
app.set("view engine", "ejs"); app.set("views", "views"); const adminRoutes =
    require("./routes/admin"); const shopRoutes = require("./routes/shop"); const
    User=require('./models/user') app.use(bodyParser.urlencoded({ extended:
    false })); app.use(express.static(path.join(__dirname, "public")));

```

```
app.use((req,res,next)=>{ User.findById("6a25536ac0e11e9bf2ae7b5d").then( user=>{ req.user=new User(user.userName,user.email,user.cart,user._id) next() } ).catch(err=>{ console.log(err) }) // next() }) app.use("/admin", adminRoutes);
app.use(shopRoutes); app.use(errorController.get404);
mongoConnect(=>{ app.listen(3000, () => { console.log('Server running on port 3000'); }); }); controllers and routes =====
```

تمام يا معلم، أول حاجة أحب أوضحك نقطة مهمة جدًا قبل ما نبدأ: **المشروع ده فيه انتقال بين طريقتين مختلفتين للتعامل مع الداتا بيز**، ودي نقطة لو فهمتها هيفرق معاك جدًا.

MongoDB وفيه Sequelize قبل الشرح: ليه فيه

أنت عندك أول 4 ملفات

- Cart
- CartItem
- Order
- OrderItem

Sequelize دول معمولين بـ

PostgreSQL و MySQL زي SQL بيستخدم غالبًا مع قواعد بيانات ORM ده Sequelize.

لكن باقي المشروع

- Product
- User
- database.js

MongoDB Native Driver شغال بـ

وبعد كده نقل المشروع لـ Sequelize بـ SQL غالبًا صاحب الكورس كان بيشرح الأول MongoDB.

دي غالبًا مش مستخدمة في النسخة الحالية Sequelize يعني ملفات

Sequelize أولاً: ملفات

models/cart.js

```
JavaScriptconstSequelize=require('sequelize')
```

يعني إيه ده؟

Modules. عنده نظام Node.js.

معناها require():

أو ملف خارجي علشان أستخدمه Package هاتلي.

Sequelize. هنا بنستورد مكتبة

```
JavaScriptconstsequelize=require('../util/database')
```

اللي بيربط التطبيق بقاعدة البيانات connection ده بيحجب الـ

يعمل اتصال لوحده، كلهم بيشاركوا نفس الاتصال Model يعني بدل ما كل

وده اسمه:

Single Database Connection

```
JavaScriptconstCart=sequelize.define('cart',{
```

دي أهم نقطة.

معناها define:

cart. جديد اسمه Model اعمل

SQL. في Table ييمثل الـ Model الـ Sequelize في

:يعني كأنك كتبت

```
SQLCREATETABLE cart
```

تعريف الأعمدة

```
JavaScriptid:{
```

```
type:Sequelize.INTEGER,
```

```
autoIncrement:true,  
allowNull:false,  
primaryKey:true  
}
```

دە معناه

id. اسم Column هنعمل

type

JavaScripttype: Sequelize.INTEGER

Integer. نوع البيانات

زري:

1
2
3

autoIncrement

JavaScriptautoIncrement:true

.يعني الرقم يزيد تلقائيًا

مثلا:

Cart: أول

id = 1

الثاني:

id = 2

وهكذا.

allowNull

JavaScriptallowNull:false

.معناها ممنوع تكون القيمة فاضية

primaryKey

JavaScriptprimaryKey:true

.يعني ده المفتاح الأساسي للجدول

.مميز ID كل صف لازم يكون له

:بعدها

JavaScript})

Model. خلصنا تعريف ال

JavaScriptmodule.exports =Cart

.علشان ملفات تانية تستخدمه Model يعني صدر ال

CartItem و OrderItem

.دول نفس الفكرة بالظبط

:بس عندهم زيادة

JavaScriptquantity: Sequelize.INTEGER

quantity. اسمه Column ده

:مثلا

cartItem

id | quantity

1 | 3

2 | 5

.يعني عدد القطع

Order

تقريبًا Cart نفس.

يمثل جدول الطلبات

MongoDB: دلوقتي ندخل على الجزء المهم

util/database.js

ده يعتبر قلب الاتصال بالداتا بيز

dns

JavaScriptconst dns=require('dns')

Node.js دي مكتبة موجودة في

DNS. وظيفتها التعامل مع

setServers

```
JavaScriptdns.setServers([  
'8.8.8.8',  
'8.8.4.4'  
])
```

DNS Servers. هنا بيحدد

Google الخاصين بـ DNS دول

ليه؟

خصوصًا في بعض الشبكات DNS بيحصل مشكلة في الـ MongoDB Atlas أحيانًا

ثابت DNS فبيحدد

MongoClient استيراد

```
JavaScriptconst { MongoClient } = require('mongodb')
```

destructuring هنا بنعمل.

:يعني بدل

```
JavaScriptconstmongodb=require('mongodb')
```

```
mongodb.MongoClient
```

.مباشرة MongoClient بنجيب

رابط قاعدة البيانات

```
JavaScriptconsturi='mongodb+srv://...'
```

Connection String ده

:فيه

- اسم المستخدم
- الباسورد
- Cluster اسم الـ
- معلومات الاتصال

.في المشاريع الحقيقية ما بنحطوش هنا

:بنحطه في

.env

.لأن ده سر

_db متغير

```
JavaScriptlet_db
```

let؟ ليه

.في البداية undefined لأنه هيكون

وبعد الاتصال:

_db = database object

mongoConnect

JavaScriptconstmongoConnect=callback => {

argument. تانية كـ Function بتاخذ function دي

ده اسمه:

Callback Function

يعني:

"لما تخلص الاتصال نفذ الكود ده".

الاتصال:

JavaScriptMongoClient.connect(uri)

Promise. بترجع Function دي

ليه؟

لأن الاتصال بالداتا بيز بياخد وقت

لو نجح:

JavaScript.then(client=>{

ال Promise اتحققت

client هو الاتصال كامل

JavaScript_db=client.db()

هنا بنجيب الداتا بيز نفسها ونخزنها

ليه؟

تقدر تستخدمها Models علشان كل

```
JavaScriptcallback()
```

بعد ما الداتا بيز بقت جاهزة نشغل السيرفر

لو حصل خطأ:

```
JavaScript.catch(err=>{  
  throwerr  
})
```

يعني وقف البرنامج واطلع الخطأ

getDb

```
JavaScriptconstgetDb= ()=>{
```

ترجع الداتا بيز Function دي

```
JavaScriptif(_db){  
  return_db  
}
```

لو الاتصال موجود رجعه

```
JavaScriptthrow'no database found'
```

Error. لو حد حاول يستخدم الداتا قبل الاتصال نرمي

التصدير

```
JavaScriptexports.mongoConnect =mongoConnect  
exports.getDb =getDb
```

يعني الملفات الثانية تقدر تعمل:

```
JavaScriptconstgetDb=require('./database').getDb
```

(أهم ملف في المشروع) app.js

دو يعتبر نقطة بداية التطبيق

```
JavaScriptconstexpress=require('express')
```

بنستورد Express.

Express فوق Node.js Web Server ييني Framework عبارة عن

```
JavaScriptconstapp=express()
```

هنا أنشأنا التطبيق

:كأنك بتقول

"ابدأ إنشاء السيرفر".

View Engine

```
JavaScriptapp.set('view engine','ejs')
```

معناه:

EJS هتكون View أي ملفات

Templates يدويًا تقدر تعمل HTML يعني بدل ما تبعت

```
JavaScriptapp.set('views','views')
```

EJS مكان ملفات الـ

Body Parser

```
JavaScriptapp.use(bodyParser.urlencoded({extended:false}))
```

لما الفورم يبعث:

```
title=phone&price=500
```

يحولها إلى Express:

```
JavaScriptreq.body = {  
  title:"phone",  
  price:"500"  
}
```

Static Files

```
JavaScriptapp.use(express.static(path.join(__dirname,'public')))
```

يسمح للمتصفح يوصل للملفات الثابتة:

- CSS
 - JS
 - Images
-

جلب المستخدم Middleware

دي من أهم الحاجات

```
JavaScriptapp.use((req,res,next)=>{
```

يدخل يمر هنا Request أي

البحث عن المستخدم

```
JavaScriptUser.findById("6a255...")
```

من الداتا بيز User يجيب Request كل

Promise: ترجع

```
JavaScript.then(user=>{
```

Async. عملية MongoDB لأن البحث في

new User؟ ليه عمل

```
JavaScriptreq.user =newUser(  
  user.userName,  
  user.email,  
  user.cart,  
  user._id  
)
```

هنا نقطة عبقرية.

لو عملنا:

```
JavaScriptreq.user =user
```

عادي Object هتحصل على.

لكن لما نعمل:

```
JavaScriptnewUser()
```

Class User من Instance هتحصل على.

:يعني يقدر يستخدم

```
JavaScriptreq.user.addToCart()  
req.user.getCart()  
req.user.addOrder()
```

Class User. وده السبب الحقيقي لكتابة

next()

```
JavaScriptnext()
```

معناها:

"اللي بعدي Middleware خلصت دوري، روح للـ".

لو نسيتهما الطلب هيقف.

Routes

```
JavaScriptapp.use("/admin",adminRoutes)
```

أي رابط يبدأ

/admin

adminRoutes. يروح للـ

```
JavaScriptapp.use(shopRoutes)
```

shopRoutes. باقي الروابط تروح للـ

404

```
JavaScriptapp.use(errorController.get404)
```

Route لو مفيش مطابق.

تشغيل السيرفر

```
JavaScriptmongoConnect(=>{  
  app.listen(3000)  
})
```

دي من أهم الأفكار

ليه مش عملنا

```
JavaScriptapp.listen()  
mongoConnect()
```

قبل ما قاعدة البيانات تكون جاهزة Request لأن السيرفر ممكن يستقبل

فإحنا بنقول

1. MongoDB اتصل بـ
2. db_ خزن الاتصال في
3. بعد النجاح شغل السيرفر

ملخص الجزء ده

أنت فهمت دلوقتي:

- MongoDB و Sequelize الفرق بين
 - Model معنى
 - define معنى
 - Primary Key و Auto Increment ال
 - MongoDB كيف يتم إنشاء اتصال
 - Promise و then(). معنى
 - mongoConnect في Callback ليه بنستخدم
 - Middleware معنى
 - req.user داخل new User ليه نعمل
 - مهمة next() لماذا
 - لماذا نبدأ السيرفر بعد نجاح الاتصال
-

بالتفصيل الشديد، وده الجزء User Model و Product Model في الرسالة الجاية هنبدأ
الأهم فعلاً في المشروع لأن فيه:

- JavaScript في Classes
- Constructor
- static methods
- MongoDB في CRUD
- insertOne, find, findOne, updateOne, deleteOne
- \$set
- ObjectId

- ال Cart logic.
- map, filter, findIndex, وال Spread Operator.

لوجدك إن شاء الله Models وده الجزء اللي هيخليك تقدر تكتب ال

كمل

:تمام يا معلم، دلوقتي داخلين على أهم جزء في المشروع كله
Product Model و User Model.

مع (Object Oriented Programming (OOP هنا هنفهم فعليًا إزاي بنطبق فكرة
MongoDB، بتتعامل مع الداتا بيز Models وإزاي ال

أولاً: Product Model

الاستيراد

```
JavaScriptconstmongodb=require('mongodb')
```

MongoDB. هنا بنستورد مكتبة

ليه؟

:لأننا محتاجين منها حاجة اسمها

```
JavaScriptObjectId
```

MongoDB. الخاص بـ ID وده نوع ال

:بيكون بالشكل ده Mongo في ID مثلا ال

```
665f0f9b8f3eab5a17a2c113
```

ObjectId. من نوع Object عادي، ده String وده مش

```
JavaScriptconstgetDb=require('../util/database').getDb
```

getDb. اسمها Function بنجيب

فاكر لما عملنا

```
JavaScriptexports.getDb =getDb
```

في database.js؟

Database Connection. دلوقتي بنستخدمها علشان نجيب ال

Class إنشاء

```
JavaScriptclassProduct {
```

Class؟ ليه عمل

Object ك Product علشان يمثل ال

Class من ال Instance جديد هيكون Product يعني كل

مثال:

```
JavaScriptconstphone=newProduct(  
"iPhone",  
1000,  
"Apple Phone",  
"image.png"  
)
```

Functions عنده خصائص و Object دلوقتي phone ال

Constructor

```
JavaScriptconstructor(title, price, description, imageUrl, id, userId)
```

:بيتنفذ تلقائيًا أول ما نعمل Constructor ال

```
JavaScriptnewProduct()
```

Object تخزين البيانات داخل

```
JavaScriptthis.title =title
```

معناها:

"الحالي Object خزن القيمة اللي جاية في الـ"

مثلا:

```
JavaScriptnewProduct("Laptop")
```

تصبح:

```
JavaScript{  
  title: "Laptop"  
}
```

نفس الكلام:

```
JavaScriptthis.price =price  
this.description =description  
this.imageUrl =imageUrl
```

الجزء المهم جدًا

```
JavaScriptif(id){  
  this._id =newmongodb.ObjectId(id)  
}
```

خلينا نفهم ليه.

عنده MongoDB كل Document في

```
JSON{  
  "_id": ObjectId("123456")  
}
```

عند إنشاء منتج جديد:

```
JavaScriptnewProduct(  
  "phone",  
  500  
)
```

ID. مفيش

ليه؟

تلقائيًا ID هو اللي هيعمل MongoDB لأن

موجود Product لكن لما نعمل تعديل

Edit Product

ID. بيبقى عندنا

فحتاج نحوله

JavaScriptnewObjectId(id)

يفهمه MongoDB علشان

بعدها:

JavaScriptthis.userId =userId

User. يعني المنتج تابع لأي

مثلا:

```
JSON{
  title: "Phone",
  userId: ObjectId("user123")
}
```

save()

Functions. دي من أهم

JavaScriptsave(){

وظيفتها:

"موجود Product جديد أو تحديث Product إما إنشاء"

:يعني هي بتعمل حاجتين

- Create
 - Update
-

Database جلب

JavaScriptconstdb=getDb()

هنا بنجيب الاتصال اللي اتخزن في

JavaScript_db

dbOp متغير

JavaScriptletdbOp

ليه عملنا متغير؟

علشان هنخزن فيه العملية اللي هتتنفذ

:مممكن يكون

insertOne

أو

updateOne

لو المنتج موجود

JavaScriptif(this._id){

يعني

ID. لو عندي

معناه

"أنا بعمل تعديل"

updateOne

```
JavaScriptdbOp=db  
.collection('products')  
.updateOne(  
  { _id: this._id },  
  { $set: this }  
)
```

خلينا نفكها

collection

```
JavaScriptdb.collection('products')
```

اسمها Collection افتح

products

SQL في Table زي

updateOne

معناها:

واحد Document حدث

Object أول:

```
JavaScript{ _id: this._id }
```

ده شرط البحث

SQL زي

```
SQLWHERE id =5
```

Object ثاني:

```
JavaScript{$set:this}
```

معناها:

غير البيانات بالقيم الجديدة

مثال:

قبل:

```
JSON{  
  title:"old",  
  price:100  
}
```

بعد:

```
JavaScriptthis= {  
  title:"new",  
  price:200  
}
```

هـيحدث القيم.

لو منتج جديد

```
JavaScriptelse {  
  dbOp=db  
  .collection('products')  
  .insertOne(this)  
}
```

معناها:

MongoDB الحالي داخل Object احفظ الـ

مثلا:

```
JavaScriptthis= {  
  title:"Phone",  
  price:500  
}
```

Document هيتحول لـ

dbOp؟ ليه رجعنا

JavaScriptreturndbOp

لأن:

insertOne

و

updateOne

Promise. بيرجعوا

:يعني نقدر نعمل

```
JavaScriptproduct.save()  
.then(result=>{  
  console.log(result)  
})
```

then()

```
JavaScript.then(result=>{  
  console.log(result)  
})
```

.يتنفذ بعد نجاح العملية

catch

```
JavaScript.catch(err=>{  
  console.log(err)  
})
```

Error. لو حصل

static fetchAll()

دي نقطة مهمة جدًا

JavaScriptstatic fetchAll()

ليه Static؟

Static عادية و Method الفرق بين

لو عندنا

JavaScriptsave()

لازم نعمل

JavaScriptconstproduct=newProduct()
product.save()

لكن

JavaScriptstatic fetchAll()

مباشرة Class بنستدعيها من الـ

JavaScriptProduct.fetchAll()

ليه؟

لأن جلب كل المنتجات مش مرتبط بمنتج معين

علشان أجيب المنتجات Product مش منطقي أعمل

الكود

JavaScriptconstdb=getDb()

جيب الاتصال

JavaScriptdb.collection('products')

Collection. افتح

find()

JavaScriptfind()

معناها:

Documents. هات كل

زوي:

SQLSELECT*FROM products;

لكن فيه نقطة مهمة.

Array. لا يرجع find()

هو يرجع حاجة اسمها:

Cursor

يعني مؤشر على نتائج البحث.

لذلك نعمل:

JavaScript.toArray()

Array. لتحويل النتائج إلى

مثلا:

قبل:

Cursor

بعد:

```
JavaScript[
  {title:"Phone"},
  {title:"Laptop"}
]
```

ثم:

```
JavaScript.then(products=>{  
  return products  
})
```

Controller. نرجع المنتجات للـ

static findById()

دي تستخدم لجلب منتج واحد.

```
JavaScript.find({  
  _id: new ObjectId(prodId)  
})
```

ID؟ ليه حولنا

ObjectId كـ `_id` يخزن MongoDB لأن

لو عملت:

```
JavaScript._id: "123"
```

لن يجد أي شيء.

next()

بعد:

```
JavaScript.find()
```

Cursor. هيطلع

لكن إحنا نريد أول نتيجة فقط.

لذلك:

```
JavaScript.next()
```

معناها:

Document هات أول

مثال:

لو النتيجة

Cursor

|

|-- Product 1

|-- Product 2

next()

تجيب أول واحد.

deleteById()

Product تحذف.

```
JavaScriptdeleteOne({  
  _id:newObjectId(prodId)  
})
```

معناها:

ID واحد بهذا ال Document احذف.

SQL زي

```
SQLDELETEFROM products WHERE id =5;
```

Product Model ملخص

الفكرة كلها كده

```
new Product()
  |
  |
  | constructor
  |
  | save()
  /  \
insert update
```

```
Product.fetchAll()
  |
  | find()
  |
  | toArray()
```

```
Product.findById()
  |
  | find()
  |
  | next()
```

```
Product.deleteById()
  |
  | deleteOne()
```

في المشروع MongoDB وقف هنا دقيقة واستوعب الجزء ده، لأنه هو أساس كل التعامل مع
وده أصعب ملف في المشروع فعلاً، لأن فيه **User Model** الرسالة الجاية هنخش على

- خطوة خطوة addToCart().
- يليه استخدمناها findIndex().
- map() في getCart().
- \$in في MongoDB.
- Spread Operator

- لل حذف filter().
- إنشاء Order من Cart.
- في كل مكان this ليه استخدم.
- Order بيتحول لـ Cart إزاي الـ.
- لحد ما الطلب يتسجل Add To Cart تدفق البيانات من أول الضغط على.

وده أهم جزء في المشروع كله.

كمل

User Model: تمام يا معلم، دلوقتي ندخل على أصعب وأهم ملف في المشروع.

Orders و Cart الملف ده هو اللي بيعمل كل منطق الـ حقيقي E-commerce Backend لو فهمته كويس هتفهم إزاي بنبي همنشي سطر سطر ومنطق بمنطق.

بداية الملف

```
JavaScriptconstmongodb=require("mongodb");
```

MongoDB بنستورد مكتبة

ليه؟

ObjectId. لأننا محتاجين منها

```
JavaScriptconstgetDb=require("../util/database").getDb;
```

اللي ترجع لنا الاتصال بالداتابيز function بنجيب الـ

فاكر؟

:عملنا database.js في

```
JavaScriptlet_db;
```

وبعد نجاح الاتصال:

```
JavaScript_db=client.db();
```

.ياخذ نفس الاتصال getDb() يطلب Model فأني

```
JavaScriptconstObjectId=mongodb.ObjectId;
```

ده اختصار

:بدل ما كل مرة نكتب

```
JavaScriptnewmongodb.ObjectId(id)
```

:نكتب

```
JavaScriptnewObjectId(id)
```

Class User إنشاء

```
JavaScriptclassUser {
```

Class؟ ليه عمل

.مش مجرد بيانات User لأن الـ

:هو عنده

- بيانات:

- o userName

- o email

- o cart

:وعنده أفعال

- يحفظ نفسه
- يضيف منتج للعربة
- يجيب منتجات العربة
- يحذف منتج

- Order يعمل

Object Oriented Programming وده هو مفهوم

Constructor

```
JavaScriptconstructor(userName, email, cart, id) {
```

لما نعمل:

```
JavaScriptnewUser(  
  "Ahmed",  
  "a@test.com",  
  {items:[]},  
  "123"  
)
```

يشغل تلقائيًا constructor الـ

تخزين القيم

```
JavaScriptthis.userName =userName;
```

يعني:

userName الحالي عنده خاصية اسمها object خلي الـ

النتيجة:

```
JavaScript{  
  userName: "Ahmed"  
}
```

```
JavaScriptthis.email =email;
```

يحفظ الإيميل

```
JavaScriptthis.cart =cart;
```

يحفظ عربة الشراء

يكون مثلاً MongoDB شكلها في

```
JavaScriptcart:{  
  items:[  
    {  
      productId:ObjectId("123"),  
      quantity:2  
    }  
  ]  
}
```

```
JavaScriptthis._id =id;
```

المستخدم ID يحفظ

save()

```
JavaScriptsave(){
```

وظيفتها حفظ مستخدم جديد في الداتايز

نجيب الاتصال

```
JavaScriptconstdb=getDb();
```

Collection users نفتح

```
JavaScriptdb.collection("users")
```

SQL في Table زي

insertOne

```
JavaScript.insertOne(this)
```

الحالي object احفظ الـ

لو كان عندنا

```
JavaScriptconst user = new User(
  "Ahmed",
  "a@test.com",
  {items: []}
);
```

ثم:

```
JavaScriptuser.save()
```

تخزن MongoDB:

```
JSON{
  "_id": ObjectId("xxxx"),
  "userName": "Ahmed",
  "email": "a@test.com",
  "cart": {
    "items": []
  }
}
```

addToCart(product)

ده من أذكى أجزاء المشروع.

الفكرة:

لو المنتج موجود في العربة:

- زود الكمية.

لو غير موجود:

- أضفه بكمية 1.
-

البحث هل المنتج موجود

```
JavaScriptconst cartProductIndex =
  this.cart.items.findIndex(cp => {
```

```
return cp.productId.toString() === product._id.toString();
});
```

نفهمها ببطء.

this.cart.items

مثلا:

```
JavaScript[
  {
    productId:111,
    quantity:2
  },
  {
    productId:222,
    quantity:1
  }
]
```

findIndex()

وترجع رقم العنصر Array تبحث داخل

مثلا:

```
JavaScript[10,20,30]
```

لو بحثنا عن 20

ترجع

1

لأن مكانه رقم 1

لو لم تجده

ترجع

-1

؟ toString() ليه

لأن:

```
JavaScriptObjectId("123")
```

String ليس

:حتى لو شكله

123

.المقارنة المباشرة قد تفشل

.لذلك نحول الاثنين إلى نص

نبدأ بكمية افتراضية

```
JavaScriptletnewQuantity=1;
```

.لو المنتج جديد سيكون عدد القطع 1

عمل نسخة من العناصر

```
JavaScriptconstupdatedCartItems= [...this.cart.items];
```

Spread Operator هنا الـ

:يعني

Array انسخ عناصر الـ

:مثلا

```
JavaScript[1,2,3]
```

:تصبح نسخة جديدة

```
JavaScript[1,2,3]
```

مباشرة؟ this.cart لماذا لا نعدل

:لأن الأفضل برمجيًا

- لا تغير الأصل مباشرة.
- اعمل نسخة ثم عدلها.

هذا يجعل الكود أوضح ويقلل المشاكل

لو المنتج موجود

```
JavaScriptif(cartProductIndex>=0)
```

يعني وجدناه

نحيب الكمية الحالية:

```
JavaScriptnewQuantity=  
this.cart.items[cartProductIndex].quantity +1;
```

مثلا:

كان:

```
quantity = 3
```

تصبح:

```
4
```

ثم نغيرها في النسخة

```
JavaScriptupdatedCartItems[cartProductIndex].quantity =  
newQuantity;
```

لو غير موجود

```
JavaScriptelse {  
updatedCartItems.push({  
productId:newObjectId(product._id),  
quantity:newQuantity
```

```
});  
}
```

نضيف عنصر جديد:

```
JavaScript{  
  productId:123,  
  quantity:1  
}
```

جديد Cart بناء

```
JavaScriptconst updatedCart= {  
  items: updatedCartItems  
};
```

النتيجة مثلاً:

```
JavaScript{  
  items:[  
    {  
      productId:123,  
      quantity:3  
    }  
  ]  
}
```

MongoDB حفظ التعديل في

```
JavaScriptreturn db  
  .collection("users")  
  .updateOne(  
    { _id: new ObjectId(this._id) },  
    { $set: { cart: updatedCart } }  
  );
```

نبحث عن المستخدم الحالي

```
JavaScript{ _id: this._id }
```

MongoDB: ثم نقول لـ

cart. غيّر فقط ال

\$set معنى

كامل Document قد تستبدل \$set بدون

لكن:

JavaScript\$set

معناها:

غير هذه الحقول فقط.

مثلا:

قبل:

```
JSON{
  name:"Ahmed",
  email:"a@test.com",
  cart:{}
}
```

بعد:

```
JavaScript$set:{
  cart:newCart
}
```

تصبح:

```
JSON{
  name:"Ahmed",
  email:"a@test.com",
  cart:newCart
}
```

الإيميل والاسم لم يتغيرا.

في كل مكان؟ this نقطة مهمة جدًا: لماذا استخدم

مثلاً:

JavaScriptthis.cart

معناها:

عربة المستخدم الحالي

لو عندنا

```
JavaScriptconstuser1=newUser("Ahmed");  
constuser2=newUser("Ali");
```

كل واحد عنده

JavaScriptuser1.cart

غير:

JavaScriptuser2.cart

الذي استدعى الدالة object تشير إلى الـ this لذلك

بالكامل Add To Cart تدفق

خلينا نربط الدنيا ببعض

المستخدم يضغط

Add To Cart

|
|
Controller

|
|
req.user.addToCart(product)

|
|
findIndex()

|
موجود؟ ----- لا

| |
أضفه زود الكمية

| |
---- updatedCart ----
|

|
updateOne + \$set
|
|
MongoDB updated

هنا نقف لأن الجزء ده لوحده فيه أفكار كثير جدًا:

- Classes
- Constructor
- this
- Arrays
- findIndex
- Spread Operator
- ObjectId
- updateOne
- \$set

User Model: في الرسالة الجاية هنكمل باقي

- (ودمج البيانات \$in و map وده فيه) getCart()
- deleteItemFromCart() و filter
- addOrder() إلى Order و تحويل الـ Cart
- getOrders()
- static findById()

وده الجزء اللي هيخليك تفهم رحلة المنتج من السلة إلى الطلب النهائي.

وهي إزاي نجيب محتويات ال User Model، تمام يا معلم، دلوقتي هنكمل أصعب حته في Order. نحذف منها، ونحولها إلى Cart.

MongoDB و JavaScript خليك مركز لأن هنا فيه شوية أفكار مهمة جدًا في

أولاً: getCart()

الكود:

```
JavaScriptgetCart() {
constdb=getDb();

constproductIds=this.cart.items.map((i) => {
returni.productId;
});

returndb
.collection("products")
.find({_id: { $in: productIds } })
.toArray()
.then(products => {
returnproducts.map(p => {
return {
...p,
quantity: this.cart.items.find(i => {
returni.productId.toString() ===p._id.toString();
}).quantity
};
});
});
}
```

الفكرة العامة

بالشكل ده Cart خلينا نتخيل إن المستخدم عنده

```
JSON{
  "items": [
    {
```

```
"productId": "111",
"quantity": 2
},
{
  "productId": "222",
  "quantity": 1
}
]
```

لاحظ حاجة مهمة جدًا

مش بيخزن بيانات المنتج كلها الـ Cart الـ

يعني مش بيخزن

```
JSON{
  title: "iPhone",
  price: 1000,
  imageUrl: "..."
```

ليه؟

Data Duplication لأن ده اسمه

يعني تكرار البيانات

Cart. تخيل عندك مليون مستخدم، وكل واحد خزن اسم المنتج والسعر والصورة داخل الـ

هيبقى عندك نسخ مكررة كتير جدًا

لذلك بنخزن بس

- productId
- quantity

المنتجات Collection ولما نحتاج تفاصيل المنتج نجيبها من

أول سطر

```
JavaScriptconst db = getDb();
```

نجيب اتصال قاعدة البيانات

المنتجات IDs استخراج

```
JavaScriptconst productIds=this.cart.items.map((i) => {  
  return i.productId;  
});
```

map؟ يعني إيه

جديدة Array وترجع Array بتمشي على كل عنصر داخل map()

مثال بسيط:

عندك:

```
JavaScript[1, 2, 3]
```

لو عملت:

```
JavaScriptnumbers.map(num => num*2)
```

هترجع:

```
JavaScript[2, 4, 6]
```

في حالتنا:

قبل:

```
JavaScriptthis.cart.items = [  
  {  
    productId: 111,  
    quantity: 2  
  },  
  {  
    productId: 222,  
    quantity: 1  
  }  
]
```

بعد map:

```
JavaScriptproductIds= [  
  111,
```

البحث عن المنتجات

JavaScriptdb.collection("products")

المنتجات Collection افتح

find

```
JavaScript.find({  
  _id: { $in: productIds }  
})
```

اسمه Operator عندنا MongoDB هنا في

JavaScript\$in

معناه:

"ده Array بتاعته موجودة داخل ال_id قيمة ال Document هات أي"

مثال:

```
JavaScriptproductIds= [111,222]
```

كأنك بتقول:

بتاعه 111 أو 222 ID هات المنتج اللي

ده شبه SQL في

```
SQLWHERE id IN (111,222)
```

Array إلى Cursor تحويل

JavaScript.toArray()

Cursor. بيرجع find() لأن

Array. إحنا عايزين

then بعد كده

JavaScript.then(products => {

ترجع المنتجات MongoDB بعد ما

شكلها ممكن يكون:

```
JavaScript[
  {
    _id:111,
    title:"Phone",
    price:500
  },
  {
    _id:222,
    title:"Laptop",
    price:1000
  }
]
```

هنا بقى الشغل المهم

JavaScriptreturnproducts.map(p => {

بنمشي على كل منتج

أول منتج:

```
JavaScriptp= {
  _id:111,
  title:"Phone"
}
```

Spread Operator

JavaScript...p

معناه:

خد كل خصائص المنتج.

يعني:

```
JavaScript{  
  _id:111,  
  title:"Phone",  
  price:500  
}
```

Cart البحث عن كمية المنتج داخل الـ

```
JavaScriptthis.cart.items.find(i => {  
  return i.productId.toString() === p._id.toString();  
})
```

find? ليه استخدمنا

.لأننا عايزين **العنصر نفسه**.

فرق مهم:

findIndex

يرجع المكان:

0
1
2

find

نفسه Object يرجع.

مثلاً:

```
JavaScript[  
  {id:1, q:3},  
  {id:2, q:5}  
]
```

لو عملت:

```
JavaScriptfind(item => item.id ===2)
```

النتيجة:

```
JavaScript{id:2, q:5}
```

ثم:

```
JavaScript.quantity
```

نجيب الكمية.

النتاج النهائي

في الآخر يرجع:

```
JavaScript[
  {
    _id:111,
    title:"Phone",
    price:500,
    quantity:2
  },
  {
    _id:222,
    title:"Laptop",
    price:1000,
    quantity:1
  }
]
```

Cart. وده شكل مناسب للعرض في صفحة الـ

ثانيًا: deleteItemFromCart()

الكود:

```
JavaScriptdeleteItemFromCart(productId){
constupdatedCartItems=this.cart.items.filter(item => {
if(!item?.productId) returnfalse;
```

```
return item.productId.toString() !== productId.toString();
});
```

الفكرة

.عايزين نحذف منتج من العربية.

عندنا:

```
JavaScript[
  {id:111, quantity:2},
  {id:222, quantity:1}
]
```

.عايزين نحذف 111.

filter

.جديدة تحتوي على العناصر التي تحقق الشرط Array تعمل filter()

مثلاً:

```
JavaScript[1,2,3,4].filter(n => n>2)
```

النتيجة:

```
JavaScript[3,4]
```

في كودنا:

```
JavaScriptreturn item.productId.toString() !== productId.toString();
```

معناها:

.بتاعها **ليس** هو المطلوب حذفه ID احتفظ بكل العناصر التي

Optional Chaining؟

JavaScriptitem?.productId

معناها:

Error. غير موجود، متعملش productId أو item لو

بذل ما يحصل:

Cannot read property 'productId'

undefined. هيرجع

تحديث المستخدم

جديدة Array بعد ما عملنا

```
JavaScriptreturndb
.collection("users")
.updateOne(
  {_id:newObjectId(this._id)},
  {$set:{cart:{items:updatedCartItems}}}
);
```

يعني:

ابحث عن المستخدم الحالي

ثم:

فقط Cart غير الـ

تدفق الحذف

Delete Button

|

Controller

|

deleteItemFromCart(id)

|
|
filter()
|
|
جديدة بدون المنتج Array
|
|
updateOne()
|
|
MongoDB Updated

ملاحظة مهمة جدًا

المنتجات Collection داخل delete لاحظ أننا لم نعمل

:يعني المنتج مازال موجود في

products

:بتاعه من Reference إحنا فقط أزلنا الـ

user.cart.items

:وده مفهوم مهم اسمه

Reference Relationship

آخر Collection لشيء موجود في ID يعني نخزن

:هنا نقف لحظة لأن الجزء ده مليان مفاهيم

- map
- find
- filter
- Spread Operator
- \$in

- Reference بين Collections

User Model: في الرسالة القادمة هنكمل آخر جزء في

- addOrder() (وده مهم جدًا)
- Order بيتحول إلى Cart إزاي ال
- then جوه then ليه عملنا
- Order بعد إنشاء Cart ليه بنفسي ال
- getOrders()
- static findById()

Routes وال Controllers بالكامل ونبدأ ندخل على ال Models وبعدها هنكون خالصنا ال .من أول الضغط على الزر لحد الداتا بيز Request وساعتها هتشوف رحلة ال

كمل

وده من أكثر الأجزاء اللي بتبين User Model، تمام يا معلم 🔥 دلوقتي هندخل على آخر جزء في حقيقي E-commerce إزاي التطبيق بيشتغل ك

:باقين Functions عندنا 3

1. addOrder()
2. getOrders()
3. static findById()

أولاً: addOrder()

الكود:

```
JavaScriptaddOrder(){  
constdb=getDb()  
returnthis.getCart().then(products=>{
```

```
const order = {
  items: products,
  user: {
    _id: new ObjectId(this._id),
    name: this.name
  }
}
return db.collection('orders').insertOne(order)
})
.then(result => {
  this.cart = { items: [] }
  return db
    .collection('users')
    .updateOne(
      { _id: new ObjectId(this._id) },
      { $set: { cart: { items: [] } } },
    );
})
}
```

الفكرة العامة

لما المستخدم يضغط

Order Now

إحنا عايزين نعمل 3 خطوات

1- Cart نجيب المنتجات الموجودة في الـ

↓

orders اسمها Collection جديد في Order ننشئ -2

↓

لأن الطلب تم بالفعل Cart نفرغ الـ -3

السطر الأول

JavaScriptconstdb=getDb()

.نجيب اتصال قاعدة البيانات

؟this.getCart()ليه استخدمنا

JavaScriptreturnthis.getCart()

:عندنا شكله كان Cart فاكّر إن الـ

```
JavaScript{
  items:[
    {
      productId: ObjectId("123"),
      quantity: 2
    }
  ]
}
```

:يعني فيه

- المنتج ID
- الكمية فقط

.المفروض يحتفظ بمعلومات المنتج كاملة وقت الشراء Order لكن الـ

:مثلاً

```
JavaScript{
  title:"iPhone",
  price:1000,
  quantity:2
}
```

ليه؟

.تخيل المستخدم اشترى موبايل سعره 1000

.بعد شهر صاحب المتجر غيّر سعر المنتج لـ 1500

.لما نعرض الطلب القديم هيطهر السعر الجديد، productId كان محتفظ فقط بـ Order لو الـ

وده خطأ

للمنتج Snapshot لذلك وقت إنشاء الطلب بنعمل

```
then(products => {})
```

```
JavaScript.then(products=>{
```

ليه then؟

لأن:

```
JavaScriptthis.getCart()
```

Promise. ترجع

Promise؟ ليه ترجع

MongoDB: على Query لأنها بتعمل

```
JavaScriptfind()
```

```
.toArray()
```

(Asynchronous) والداتا بيز عملية غير فورية

Order إنشاء ال

```
JavaScriptconst order = {  
  items: products,  
  user: {  
    _id: new ObjectId(this._id),  
    name: this.name  
  }  
}
```

بالشكل Object هيكون عندنا

```
JavaScript{  
  items: [  
    {  
      title: "Phone",  
      price: 500,  
      quantity: 2  
    }  
  ],  
}
```

```
user:{
  _id:"abc123",
  name:"Ahmed"
}
```

⚠ ملاحظة مهمة في الكود

عندك هنا:

```
JavaScriptname:this.name
```

عندك constructor في الـ

```
JavaScriptthis.userName =userName
```

اسمها Property يعني مفيش

```
JavaScriptthis.name
```

في الكود Bug فغالبا ده

الصحيح غالبا يكون

```
JavaScriptname:this.userName
```

دي نقطة ممتاز إنك لاحظتها لو كنت بتراجع الكود

Order حفظ الـ

```
JavaScriptreturndb.collection('orders').insertOne(order)
```

اسمها Collection نفتح

orders

جديد Document ونضيف

مثلاً هيبقى عندنا

orders

```
{
  _id: ObjectId("777"),

  items:[
    {
      title:"Laptop",
      price:1000,
      quantity:1
    }
  ],

  user:{
    _id:ObjectId("555"),
    name:"Ahmed"
  }
}
```

بعد نجاح إنشاء الطلب

بعد ما

JavaScriptinsertOne()

التاني then يخلص بنروح للـ

JavaScript.then(result=>{

تاني؟ then ليه فيه

لأن:

JavaScriptinsertOne()

Promise. برضه بيرجع

إحنا بنقول

"اتحفظ، اعمل الخطوة اللي بعدها Order بعد ما تتأكد إن الـ"

Object داخل الـ Cart تفريغ الـ

```
JavaScriptthis.cart = { items: [] }
```

فقط الـ Memory الموجود في الـ User ده يغير الـ

يعني:

قبل:

```
JavaScriptthis.cart = {  
  items:[  
    {productId:123, quantity:2}  
  ]  
}
```

بعد:

```
JavaScriptthis.cart = {  
  items:[]  
}
```

بعد الـ MongoDB لكن ده لم يغير.

تحديث المستخدم في قاعدة البيانات

عشان كده عملنا:

```
JavaScriptreturn db  
.collection("users")  
.updateOne(  
  { _id:newObjectId(this._id)},  
  { $set:{ cart:{items:[] } } }  
);
```

يعني:

ابحث عن المستخدم الحالي.

ثم:

فقط واجعله فارغاً غير الـ Cart

كامل addOrder() تدفق

Order Button



Controller



req.user.addOrder()



getCart()



يجلب المنتجات + الكميات



إنشاء order Object



insertOne في orders



MongoDB في cart تفرغ



Order Completed

ثانيًا: getOrders()

الكود:

```
JavaScriptgetOrders(){
constdb=getDb()
returndb
.collection('orders')
.find({'user._id':newObjectId(this._id)})
.toArray()
}
```

الفكرة

.عايزين نجيب كل الطلبات الخاصة بالمستخدم الحالي

`collection('orders')`

الطلبات Collection افتح

`find`

```
JavaScriptfind({  
'user._id': newObjectId(this._id)  
})
```

MongoDB. دي نقطة جميلة في

:بالشكل Order إحنا عندنا الـ

```
JavaScript{  
  user:{  
    _id:ObjectId("123"),  
    name:"Ahmed"  
  }  
}
```

Object. جواه Object عايزين نبحت داخل

:بنستخدم

```
JavaScript'user._id'
```

:النقطة معناها

_id ثم هات user ادخل داخل

:زري

```
JavaScriptorder.user._id
```

لماذا `toArray()`؟

:لأن

JavaScriptfind()

Cursor. يرجع

.من الطلبات Array ونحن نريد

النتيجة:

```
JavaScript[
  {
    items:[...],
    user:{...}
  },
  {
    items:[...],
    user:{...}
  }
]
```

ثالثًا: static findById()

الكود:

```
JavaScriptstatic findById(userId){
  constdb=getDb();

  returndb
    .collection("users")
    .findOne({
      _id:newObjectId(userId)
    })
    .then(user=>{
      console.log(user);
      returnuser;
    })
    .catch(err=>{
      console.log(err);
    });
}
```

static ؟ليه

أصلاً User بدون أن يكون عندنا User لأننا نريد البحث عن

:عملنا app.js مثلاً في

```
JavaScriptUser.findById("123")
```

:ولم نعمل

```
JavaScriptconstuser=newUser()
```

.لأننا لا نملك المستخدم بعد

findOne()

:استخدمنا Product في

```
JavaScriptfind().next()
```

:هنا استخدم

```
JavaScriptfindOne()
```

:الاثنتين يؤدوا نفس الفكرة تقريباً

واحد Document هات

.أبسط عندما نعرف أننا نريد عنصر واحد فقط findOne لكن

ID؟ لماذا نحول الـ

```
JavaScriptnewObjectId(userId)
```

:بهذا النوع _id يخزن MongoDB لأن

ObjectId

String. وليس

ثم:

```
JavaScript.then(user=>{  
  returnuser;  
})
```

Middleware أو Controller بعد نجاح البحث يرجع المستخدم للـ

ربطها مع app.js

فأكر الجزء ده:

```
JavaScriptUser.findById("6a255...")  
.then(user=>{  
  req.user =newUser(  
    user.userName,  
    user.email,  
    user.cart,  
    user._id  
  );  
  
  next();  
})
```

الرحلة كالتالي:

يدخل Request

↓

Middleware

↓

User.findById()

↓

MongoDB يرجع Document

↓

إنشاء User Object



له Methods إضافة



يصبح جاهز req.user



يقدر يعمل Controller أي:

req.user.addToCart()

req.user.getCart()

req.user.addOrder()

req.user.getOrders()

بالكامل User ملخص ملف

(موجود User تحتاج) عادية Methods

وظيفتها	Method
إنشاء مستخدم جديد	save()
إضافة منتج أو زيادة كميته	addToCart()
جلب منتجات السلة مع التفاصيل	getCart()
حذف منتج من السلة	deleteItemFromCart()
تحويل السلة إلى طلب	addOrder()
جلب طلبات المستخدم	getOrders()

Static Methods

Method	وظيفتها
findById()	Object بدون إنشاء User البحث عن

وفهمنا **app.js + Database + كلها Models** وبكده إحنا خلصنا الـ

- Classes و this
- Constructor
- Static Methods
- Promises و .then
- CRUD في MongoDB
- insertOne
- find, findOne
- updateOne
- deleteOne
- \$set
- \$in
- map, find, filter
- Spread Operator
- Products بالـ Cart ربط الـ
- Order إلى Cart تحويل

هتبقى ممتعة جدًا، لأنك هتشوف رحلة **Routes والـ Controllers** المرحلة الجاية لما تبعت الـ
كاملة الـ Request الـ

Browser

↓

Route

↓

Controller
↓
Model
↓
MongoDB
↓
Controller
↓
View(EJS)
↓
Browser

وده المكان اللي كل الأجزاء اللي فهمناها هتتركب فيه مع بعض 🔥

```
contoller/admin  const Product = require('../models/product');
exports.getAddProduct = (req, res, next) => { res.render('admin/edit-product',
  { pageTitle: 'Add Product', path: '/admin/add-product', editing: false }); };
exports.postAddProduct = (req, res, next) => { const title = req.body.title; const
imageUrl = req.body.imageUrl; const price = req.body.price; const description =
  req.body.description; const product=new
  Product(title,price,description,imageUrl,null,req.user._id)
  product.save() .then(result=>{ // console.log(result) console.log('create')
    res.redirect('/admin/products') }).catch(err=>{console.log(err)}) };
exports.getEditProduct = (req, res, next) => { const editMode = req.query.edit; if
  (!editMode) { return res.redirect('/'); } const prodId = req.params.productId;
  Product.findById(prodId) .then( product => { if (!product) { return
  res.redirect('/'); } res.render('admin/edit-product', { pageTitle: 'Edit Product',
    path: '/admin/edit-product', editing: editMode, product:
  product }); }).catch(err=>{ console.log(err) }); exports.postEditProduct =
  (req, res, next) => { const prodId = req.body.productId; const updatedTitle =
  req.body.title; const updatedPrice = req.body.price; const updatedImageUrl =
  req.body.imageUrl; const updatedDesc = req.body.description; const
  product=new
  Product(updatedTitle,updatedPrice,updatedDesc,updatedImageUrl,prodId)
  product.save() .then( result=>{ console.log('updated product')
  res.redirect('/admin/products'); } ).catch(err=>{ console.log(err) });
  exports.getProducts = (req, res, next) => { Product.fetchAll() //
  Product.findAll() .then(products => { res.render('admin/products', { prods:
```

```

        products, pageTitle: 'Admin Products', path:
        '/admin/products' }); }).catch(err=>{ console.log(err) }) };
exports.postDeleteProduct = (req, res, next) => { const prodId =
        req.body.productId;
        Product.deleteById(prodId) .then(()=>{ console.log('destroyed')
        res.redirect('/admin/products'); }) .catch(err=>{ console.log(err) }) };
===== controller/shop const Product = require('../models/product'); // const
        Cart = require('../models/cart'); exports.getProducts = (req, res, next) =>
        { Product.fetchAll().then(products=>{ res.render('shop/product-list', { prods:
        products, pageTitle: 'All Products', path:
        '/products' }); }).catch(err=>{ console.log(err) }) .catch(err =>
        console.log(err)); }); exports.getProduct = (req, res, next) => { const prodId =
        req.params.productId;
        Product.findById(prodId).then( product=>{ res.render('shop/product-detail',
        { product: product, pageTitle: product.title, path:
        '/products' }); } ).catch(err=>{ console.log(err) }) }; exports.getIndex = (req,
        res, next) => { Product.fetchAll().then(products=>{ res.render('shop/index',
        { prods: products, pageTitle: 'Shop', path:
        '/' }); }).catch(err=>{ console.log(err) }) .catch(err => console.log(err)); });
        exports.getCart = (req, res, next) =>
        { req.user.getCart() .then(products=>{ res.render('shop/cart', { path: '/cart',
        pageTitle: 'Your Cart', products:
        products }); }) .catch(err=>console.log(err)) }; exports.postCart = (req, res,
        next) => { const prodId = req.body.productId;
        Product.findById(prodId) .then(product=>{ return req.user.addToCart(product)
        }) .then(result=>{ console.log(result) res.redirect('/cart') }) // let
        fetchedCart // let newQuantity=1 // req.user.getCart().then(cart=>{ //
        fetchedCart=cart // return cart.getProducts({where:
        {id:prodId}}) // } // ).then(products=>{ // let product // if(products.length>0){ //
        product=products[0] // } // if(product){ // const
        oldQuantity=product.cartItem.quantity // newQuantity=oldQuantity+1 // return
        product // } // return Product.findById(pk) // }) // .then( product=>{ //
        return fetchedCart.addProduct(product,{through:
        {quantity:newQuantity}}) // }) // .then(()=>{ //
        res.redirect('/cart') // } // ) // .catch(err=>{ // console.log(err) // }) };
        exports.postCartDeleteProduct = (req, res, next) => { const prodId =
        req.body.productId;
        req.user.deleteItemFromCart(prodId).then(result=>{
        res.redirect('/cart'); }) .catch(err=>{ console.log(err) }) };

```

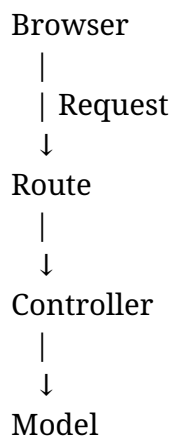
```

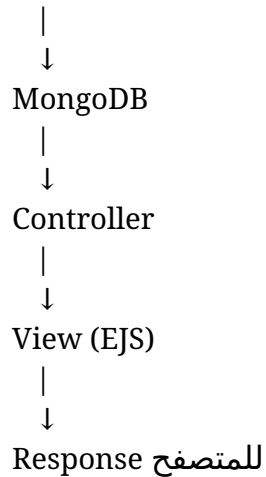
exports.postOrder=(req,res,next)=>{ let fetchedOrder
                                req.user.addOrder() .then(result=>{
res.redirect('/orders') }) .catch(err=>{ console.log(err) }) } exports.getOrders
= (req, res, next) => { req.user.getOrders({include:['products']}) .then(orders=>{
    res.render('shop/orders',{ path: '/orders', pageTitle: 'Your Orders',
    orders:orders }); }) .catch(err=>{ console.log(err) }) }; =====
routes/admin const path = require('path'); const express = require('express');
    const adminController = require('../controllers/admin'); const router =
    express.Router(); // /admin/add-product => GET router.get('/add-product',
    adminController.getAddProduct); // /admin/products => GET
router.get('/products', adminController.getProducts); // /admin/add-product =>
    POST router.post('/add-product', adminController.postAddProduct);
    router.get('/edit-product/:productId', adminController.getEditProduct);
    router.post('/edit-product', adminController.postEditProduct);
    router.post('/delete-product', adminController.postDeleteProduct);
module.exports = router; ===== routes/shop const path = require('path');
    const express = require('express'); const shopController =
    require('../controllers/shop'); const router = express.Router(); router.get('/',
    shopController.getIndex); router.get('/products', shopController.getProducts);
router.get('/products/:productId', shopController.getProduct); router.get('/cart',
    shopController.getCart); router.post('/cart', shopController.postCart);
    router.post('/cart-delete-item', shopController.postCartDeleteProduct);
    router.post('/create-order', shopController.postOrder); router.get('/orders',
    shopController.getOrders); module.exports = router;

```

تمام يا معلم 🔥🔥 دلوقتي إحنا دخلنا على المرحلة اللي هتجمع كل اللي فهمناه قبل كده

اللي المشروع ماشي عليها MVC قبل ما نمسك الكود، لازم تفهم **معمارية الـ**





"يقول: "لما حد يدخل الرابط ده نروح لمين؟ **Route** يعني الـ"

"يقول: "نعمل إيه؟ **Controller** الـ"

".يقول: "هات أو خزن البيانات **Model** الـ".

الأول (لأنها باب الدخول) Routes نبدأ بـ

routes/admin.js

الاستيراد

```
JavaScriptconst express=require('express');
```

Express الخاص بـ Router علشان نستخدم

```
JavaScriptconst adminController=require('../controllers/admin');
```

Controller بنجيب الـ

ليه؟

Logic مش المفروض يحتوي Route لأن الـ

غلط:

```
JavaScriptrouter.get('/products',(req,res)=>{  
  // Query Database  
  // Render View  
});
```

ليه؟

لأن الملف هيكبر جدًا

الأصح:

```
JavaScriptrouter.get('/products', adminController.getProducts);
```

يعني:

"Controller موجودة في Function روح نفذ Request لما يحصل"

Router إنشاء

```
JavaScriptconst router=express.Router();
```

App دي نسخة صغيرة من الـ

يعني بدل ما نعمل:

```
JavaScriptapp.get()  
app.post()
```

في كل الملفات

منفصل Router نعمل

Add Product Page

```
JavaScriptrouter.get('/add-product', adminController.getAddProduct);
```

نفهمها:

GET /admin/add-product

|
↓

adminController.getAddProduct

مش مكتوبة؟ /admin ليه

app.js فاكر في

```
JavaScriptapp.use('/admin', adminRoutes);
```

تلقائيًا /admin يضيف Express يعني

عرض المنتجات للإدارة

```
JavaScriptrouter.get('/products', adminController.getProducts);
```

الرابط النهائي:

/admin/products

إضافة منتج

```
JavaScriptrouter.post('/add-product', adminController.postAddProduct);
```

لما الفورم يعمل:

HTML<formmethod="POST">

ييجي هنا Request الـ

تعديل منتج

```
JavaScriptrouter.get('/edit-product/:productId', adminController.getEditProduct);
```

هنا عندنا حاجة اسمها:

Route Parameter

مثلاً:

/admin/edit-product/665abc123

يخزن الجزء ده في Express:

```
JavaScriptreq.params.productId
```

حفظ التعديل

```
JavaScriptrouter.post('/edit-product', adminController.postEditProduct);
```

Save. بعد ما المستخدم يعدل البيانات ويضغط

حذف منتج

```
JavaScriptrouter.post('/delete-product', adminController.postDeleteProduct);
```

Router تصدير

```
JavaScriptmodule.exports =router;
```

يستخدمه app.js علشان

Controller/admin.js دلوقتي

Logic. ده المكان اللي فيه ال

Product استيراد

```
JavaScriptconstProduct=require('../models/product');
```

ليه؟

Model. عن طريق ال MongoDB علشان تتعامل مع

1) getAddProduct

```
JavaScriptexports.getAddProduct = (req,res,next)=>{
```

:ليه عملنا

JavaScriptexports.getAddProduct

بدل:

JavaScriptfunctiongetAddProduct()

كثيرة من نفس الملف Functions لأننا عايزين نصدر

وظيفتها:

عرض صفحة إضافة منتج.

JavaScriptres.render('admin/edit-product',{

معنى render:

"للمتصفح HTML وابتعت EJS شغل ملف"

JavaScriptpageTitle:'Add Product'

View. متغير نرسله للـ

نقدر نعمل EJS في

ejs<title><%= pageTitle %></title>

JavaScriptediting:false

ليه؟

لأن نفس الصفحة تستخدم للإضافة والتعديل

لو:

editing = false

Add. يبقى

لو:

editing = true

Edit. يبقى

:وده اسمه

Reusable View

.يعني نفس الصفحة لها أكثر من استخدام

2) postAddProduct

Add Product لما المستخدم يضغط

قراءة البيانات

JavaScriptconsttitle=req.body.title;

فاكر Body Parser؟

:هو اللي عمل

JavaScriptreq.body

:مثلاً الفورم

HTML<inputname="title">

:تصبح

JavaScriptreq.body.title

جديد Product إنشاء

```
JavaScriptconstproduct=newProduct(
title,
price,
description,
imageUrl,
null,
req.user._id
);
```

خلينا نفكها.

كان Constructor الـ

```
JavaScriptconstructor(  
  title,  
  price,  
  description,  
  imageUrl,  
  id,  
  userId  
)
```

إذن:

title → اسم المنتج
price → السعر
description → الوصف
imageUrl → الصورة
id → null
userId → المستخدم

؟ id = null لماذا

لأننا ننشئ منتج جديد.

ID؟ ومين هيعمل

MongoDB تلقائيًا.

؟ req.user._id لماذا

علشان نربط المنتج بصاحبه.

في قاعدة البيانات:

```
JSON{  
  title:"Phone",  
  price:500,  
  userId:ObjectId("123")  
}
```

الحفظ

JavaScriptproduct.save()

Product في save فاكّر

كان يعمل:

id: لو يوجد

Update

لو لا يوجد

Insert

id. هنا لا يوجد

إذن:

insertOne()

ثم

JavaScript.then(result=>{

بعد نجاح الحفظ.

JavaScriptres.redirect('/admin/products')

يعني:

اطلب من المتصفح فتح صفحة المنتجات.

3) getProducts

الرابط:

GET /admin/products

Model: استدعاء

```
JavaScriptProduct.fetchAll()
```

Static؟ له

معين Product لأننا لا نحتاج

نريد كل المنتجات

Product: داخل

```
collection('products')
```

```
|
```

```
find()
```

```
|
```

```
toArray()
```

بعد رجوع البيانات:

```
JavaScript.then(products=>{
```

شكلها:

```
JavaScript[
```

```
{title:"Phone"},
```

```
{title:"Laptop"}]
```

نعرضها:

```
JavaScriptres.render('admin/products',{
```

```
prods:products
```

```
})
```

EJS: في الـ

```
ejs<%= prods[0].title %>
```

4) getEditProduct

دي فيها شوية أفكار مهمة.

أولاً:

```
JavaScriptconst editMode=req.query.edit;
```

دي Query Parameter.

مثلاً الرابط:

```
/admin/edit-product/123?edit=true
```

يتقسم إلى:

Path Parameter:

123

↓

```
req.params.productId
```

و:

Query Parameter:

edit=true

↓

```
req.query.edit
```

لماذا عمل هذا الشرط؟

```
JavaScriptif(!editMode){  
  returnres.redirect('/');  
}
```

edit=true يعني لو حد دخل رابط التعديل بدون

ارجعه للرئيسية.

ID جلب

```
JavaScriptconst prodId=req.params.productId;
```

مثلاً:

/edit-product/888

تصبح:

prodId = 888

البحث عن المنتج

JavaScriptProduct.findById(prodId)

MongoDB يروح لـ

```
JavaScriptfind({_id:ObjectId(prodId)})  
.next()
```

لو لم يجده

```
JavaScriptif(!product){  
returnres.redirect('/');  
}
```

لو وجده

```
JavaScriptres.render('admin/edit-product',{  
  editing:true,  
  product:product  
});
```

لاحظ حاجة عبقرية هنا

View نفس

edit-product.ejs

تستخدم في

- Add Product
- Edit Product

والفرق فقط

لحد هنا نقف

ليه؟

.لأن الرسالة دي وصلت لجزء كبير، ومهم إنك تستوعب الفكرة

:إحنا غطينا

- MVC Architecture
- Route
- Router
- GET vs POST
- Route Parameters
- Query Parameters
- res.render
- res.redirect
- req.body
- req.params
- req.query
- ربط Product ب User
- للإضافة والتعديل View لماذا نستخدم نفس الـ

adminController: في الرسالة القادمة هنكمل باقي

- save() وهنا هتشوف حركة ذكية في postEditProduct()
- postDeleteProduct()
- shopController وبعدها ندخل على
عرض المنتجات 0

- o Product Details
- o Cart Flow
- o Add To Cart من أول الضغط لحد MongoDB
- o Orders Flow

🔥 والجزء الجاي هو اللي هيجمع المشروع كله من أول المستخدم لحد قاعدة البيانات

كمل

.تمام يا معلم 🔥 نكمل من حيث وقفنا

إحنا خلصنا:

- Admin بتاعة Routes
- getAddProduct
- postAddProduct
- getProducts
- getEditProduct

adminController: فاضل في

1. postEditProduct()
2. postDeleteProduct()

shopController. وبعدين ندخل على

أولاً: postEditProduct()

الكود:

```
JavaScriptexports.postEditProduct = (req, res, next) => {
```

:هتشتغل لما الفورم بتاع تعديل المنتج يعمل Function دي

POST /admin/edit-product

Save. عدل البيانات، ضغط، Edit يعني المستخدم فتح صفحة

جلب البيانات من الفورم

```
JavaScriptconst prodId=req.body.productId;
const updatedTitle=req.body.title;
const updatedPrice=req.body.price;
const updatedImageUrl=req.body.imageUrl;
const updatedDesc=req.body.description;
```

Form. هنا بنقرأ البيانات اللي جاية من الـ

HTML: مثلاً عندنا

```
HTML<input name="title">
<input name="price">
```

لما المستخدم يكتب:

```
title = iPhone
price = 1000
```

:يعمل (bodyParser عن طريق) Express

```
JavaScriptreq.body = {
  title: "iPhone",
  price: 1000
}
```

.فنقدر نجيب القيم

جديد Product إنشاء

```
JavaScriptconst product=new Product(
  updatedTitle,
  updatedPrice,
  updatedDesc,
  updatedImageUrl,
  prodId
)
```

هنا في حاجة مهمة جدًا

أنت ممكن تسأل:

وهو عايز يعدل منتج موجود؟ مش المفروض يجيب المنتج new Product هو ليه بيعمل الأول؟

سؤال ممتاز

بتاعنا اتصمم بطريقة ذكية Model السبب إن الـ

Product.save()؟ فاكّر

كان عندنا

```
JavaScriptif(this._id){  
  updateOne()  
}else{  
  insertOne()  
}
```

يعني:

ID: عنده Product لو الـ

Update تعديل

ID: لو مفيش

Insert إضافة

بما إننا مررنا

JavaScriptprodId

عمل constructor فالـ

JavaScriptthis._id =newObjectId(prodId)

وبالتالي لما نعمل

JavaScriptproduct.save()

هيعرف إن ده تعديل

تنفيذ الحفظ

```
JavaScriptproduct.save()  
.then(result=>{
```

هنا Promise.

يعني:

"تخلص التعديل MongoDB استنى"

بعد النجاح:

```
JavaScriptconsole.log('updated product')
```

للتأكد أثناء التطوير.

ثم:

```
JavaScriptres.redirect('/admin/products');
```

يرجع المستخدم لقائمة المنتجات.

رحلة تعديل المنتج بالكامل

User Click Edit



GET /edit-product/:id



عرض الفورم بالبيانات القديمة



User changes data



POST /edit-product



new Product(id + data)

|
↓
save()
|
↓
if (_id)
|
↓
updateOne()
|
↓
MongoDB Updated

ثانيًا: postDeleteProduct()

الكود:

```
JavaScriptexports.postDeleteProduct = (req,res,next)=>{
```

Delete. دي تشتغل لما المستخدم يضغط زر

ID جلب

```
JavaScriptconstprodId=req.body.productId;
```

في الفورم Hidden Input غالبًا عندك:

```
HTML<inputtype="hidden"name="productId">
```

المنتج ID علشان تبعت.

الحذف

```
JavaScriptProduct.deleteById(prodId)
```

Static Method. فاكر إنها

Static؟

لأننا مش محتاجين نعمل:

JavaScriptnewProduct()

علشان نحذف.

فقط ID إحنا عندنا

يحصل Product Model داخل:

```
collection('products')  
  |  
  ↓  
deleteOne({_id:ObjectId(id)})
```

بعد نجاح الحذف

JavaScript.then()=>{

يعني:

قالت MongoDB:

تم الحذف بنجاح

ثم:

JavaScriptres.redirect('/admin/products');

يرجع لصفحة المنتجات

ملخص Admin Controller

خلينا نربط كل حاجة

إضافة منتج

POST /add-product

|
↓

```
req.body
  ↓
new Product()
  ↓
save()
  ↓
insertOne()
```

تعديل منتج

```
POST /edit-product
  ↓
new Product(id)
  ↓
save()
  ↓
updateOne()
```

حذف منتج

```
POST /delete-product
  ↓
deleteById()
  ↓
deleteOne()
```

نقطة تصميم مهمة جدًا

Product Model أنت دلوقتي ممكن تلاحظ حاجة جميلة في

واحدة Function إحنا عندنا

JavaScriptsave()

:يتعمل حاجتين

Create + Update

:مشهور اسمه Design وده

Upsert Pattern (بشكل يدوي)

ID. تحدد هل تنشئ ولا تحدث حسب وجود ال Method يعني نفس ال

ملاحظة احترافية في الكود

في postEditProduct:

```
JavaScriptconstproduct=newProduct(  
updatedTitle,  
updatedPrice,  
updatedDesc,  
updatedImageUrl,  
prodId  
)
```

userId. لاحظ إننا لم نمرر

:يعني لما يحصل

JavaScript\$set: this

في save()

حسب شكل البيانات الموجودة في قاعدة Product من ال userId قد يتم حذف أو تغيير حقل البيانات وطريقة التحديث.

أيضًا أو نحدد الحقول المسموح userId في مشروع حقيقي غالبًا نجيب المنتج القديم أو نمرر بتعديلها فقط.

دي من النقاط اللي لازم تبقى واخد بالك منها.

🦵 بالكامل Admin كده خلصنا

Shop Controller في الرسالة القادمة هنبداً

وده الجزء اللي هتشوف فيه التطبيق وهو شغال فعلياً

- عرض كل المنتجات
- Product Details عرض
- Home Page عرض
- Cart جلب الـ
- req.user الشغل إزاي
- MongoDB لحد View من الـ Add To Cart رحلة زر

🔥 اللي فهمناها ببعض Models وده الجزء اللي هيربط كل الـ

كمل كل اللي فاضل مرة واحدة

تمام يا معلم 🔥 دلوقتي هنخلص كل اللي باقي في المشروع مرة واحدة

ربط كل حاجة ببعض من أول ما المستخدم يضغط زر لحد + shopRoutes + shopController الداتا بيز

routes/shop.js: أول حاجة

ده باب دخول صفحات المتجر

```
JavaScriptrouter.get('/', shopController.getIndex);
```

معناها:

لما حد يدخل

GET /

:روح نفذ

JavaScriptshopController.getIndex

Home يعني صفحة ال

عرض كل المنتجات

JavaScriptrouter.get('/products', shopController.getProducts);

لما المستخدم يدخل

/products

روح هات المنتجات واعرضها

عرض منتج واحد

JavaScriptrouter.get('/products/:productId', shopController.getProduct);

Route Parameter هنا عندنا

مثلاً:

/products/685123abc

يخزن الجزء ده هنا Express:

JavaScriptreq.params.productId

Cart صفحة ال

JavaScriptrouter.get('/cart', shopController.getCart);

يعرض سلة المشتريات

إضافة منتج للعربة

JavaScriptrouter.post('/cart', shopController.postCart);

لما تضغط

Add To Cart

هنا POST الفورم يعمل

Cart حذف منتج من

```
JavaScriptrouter.post('/cart-delete-item', shopController.postCartDeleteProduct);
```

Order إنشاء

```
JavaScriptrouter.post('/create-order', shopController.postOrder);
```

لما تضغط:

Order Now

عرض الطلبات

```
JavaScriptrouter.get('/orders', shopController.getOrders);
```

يعرض الطلبات السابقة.

shopController ندخل على

في البداية:

```
JavaScriptconst Product = require('../models/product');
```

ليه؟

علشان نتعامل مع المنتجات.

1) getProducts

```
JavaScriptexports.getProducts = (req, res, next) => {  
  Product.fetchAll()
```

Model. نطلب كل المنتجات من

فاكر:

```
Product.fetchAll()
  |
  find()
  |
  toArray()
```

Promise. ترجع

لما تنجح:

```
JavaScript.then(products => {
  res.render('shop/product-list',{
    prods: products
  });
})
```

المنتجات Array نعرض صفحة المنتجات ونبعث لها.

مثلاً EJS داخل:

```
ejs<%= prods[0].title %>
```

ملاحظة مهمة في الكود

عندك هنا:

```
JavaScript.catch(err=>{
  console.log(err)
})
```

```
.catch(err => console.log(err));
```

مكرر catch فيه.

ده خطأ في الكود.

واحد فقط Catch المفروض يكون.

2) getProduct

دي صفحة تفاصيل منتج واحد.

ID: أولاً نجيب الـ

```
JavaScriptconst prodId=req.params.productId;
```

مثلاً:

```
/products/123
```

تصبح:

```
JavaScriptprodId=123
```

نبحث عنه

```
JavaScriptProduct.findById(prodId)
```

MongoDB: يروح

products collection

|

find

|

next()

بعد رجوع المنتج:

```
JavaScriptres.render('shop/product-detail',{  
  product: product  
});
```

نعرض التفاصيل.

3) getIndex

دي الصفحة الرئيسية.

getProducts تقريباً نفس

تعمل:

JavaScriptProduct.fetchAll()

وترسل النتائج إلى:

shop/index.ejs

ملاحظة

فيها نفس المشكلة

JavaScript.catch()
.catch()

مكرر Catch.

4) getCart

Functions. دي من أهم

الكود:

JavaScriptreq.user.getCart()

app.js. في Middleware هنا بقى تظهر فائدة الـ

فاكر:

JavaScriptreq.user = newUser(...)

جاهز User عنده Controller يعني أي.

بديل ما نكتب:

ابحث عن المستخدم
Cart ثم هات

كل مرة.

نكتب فقط:

JavaScriptreq.user.getCart()

يُحصل User Model داخل:

```
cart.items
  |
  |
  | productIds استخراج
  |
  | $in باستخدام find
  |
  | quantity دمج بيانات المنتج مع
```

يرجع لنا شكل مثل:

```
JavaScript[
{
  title:"Phone",
  price:1000,
  quantity:2
}
]
```

ثم:

```
JavaScriptres.render('shop/cart',{
  products: products
})
```

يعرض العربة.

بالكامل Cart رحلة فتح

GET /cart

|

v

Route

|
v

shopController.getCart

|
v

req.user.getCart()

|
v

User Model

|
v

MongoDB

|
v

EJS View

5) postCart

دي لما المستخدم يضغط

Add To Cart

المنتج ID أولاً نجيب

```
JavaScriptconst prodId=req.body.productId;
```

نبحث عن المنتج

JavaScriptProduct.findById(prodId)

ليه؟

المنتج Object محتاجة addToCart لأن

بعد ما يرجع:

JavaScriptreturnreq.user.addToCart(product)

User Model يدخل

هناك يحصل:

هل المنتج موجود؟

|
لا

نعم

|
أضف عنصر جديد quantity زود

|
updateOne للمستخدم

بعد النجاح:

JavaScriptres.redirect('/cart')

يرجع المستخدم يشوف السلة

كاملة Add To Cart رحلة

Click Add To Cart

|
v

POST /cart

|
v

Controller

|
v

Product.findById()

|
v

req.user.addToCart()

|
v

update users collection

|
v

Redirect /cart

6) postCartDeleteProduct

Delete. لما تضغط

ID: نأخذ

JavaScriptconstprodId=req.body.productId;

ثم:

JavaScriptreq.user.deleteItemFromCart(prodId)

Model: داخل

filter()

|

إزالة العنصر

|

updateOne()

ثم:

JavaScriptres.redirect('/cart');

7) postOrder

لما المستخدم يضغط:

Order Now

ينفذ:

JavaScriptreq.user.addOrder()

User Model: داخل

getCart()

|

إنشاء order object

|

insertOne في orders

|

تفريغ cart

بعدها:

```
JavaScriptres.redirect('/orders')
```

ملاحظة صغيرة

عندك:

```
JavaScriptletfetchedOrder
```

لكن لم يتم استخدامه

مممكن حذفه.

القديمة Sequelize ده بقايا من نسخة

8) getOrders

الكود:

```
JavaScriptreq.user.getOrders()
```

User Model: في

```
JavaScriptfind({  
'user._id': this._id  
})
```

يعني:

هات الطلبات الخاصة بالمستخدم الحالي.

ثم:

```
JavaScript.toArray()
```

Array. ترجع

ثم:

```
JavaScriptres.render('shop/orders',{  
  orders: orders  
});
```

تعرض الطلبات

ملاحظة مهمة جدًا

في كودك مكتوب:

```
JavaScriptreq.user.getOrders({include:['products']})
```

MongoDB. وده غلط مع نسخة

ليه؟

:معرف كده getOrders لأن

```
JavaScriptgetOrders(){  
}
```

Arguments. ولا يأخذ أي

للعلاقات بين الجداول Sequelize ده كان مستخدم في include

:الصحيح يكون

```
JavaScriptreq.user.getOrders()
```

الصورة الكاملة للمشروع

إضافة منتج

Admin Form

|
|

POST /admin/add-product

|
|

new Product()

|

|

save()

|

|

insertOne(products)

تعديل منتج

Edit Form

|

|

POST /edit-product

|

|

new Product(id)

|

|

save()

|

|

updateOne()

إضافة للعربة

Product Page

|

|

POST /cart

|

|

Product.findById()

|

|

User.addToCart()

|

|

users collection

إنشاء Order

Cart

|

|

POST /create-order

|

|

User.addOrder()

|

|

orders collection

|

|

Clear Cart

العلاقات بين ال Collections

users

```
JSON{
  "_id": "1",
  "userName": "Ahmed",
  "cart": {
    "items": [
      {
        "productId": "10",
        "quantity": 2
      }
    ]
  }
}
```

products

```
JSON{
  "_id": "10",
  "title": "iPhone",
  "price": 1000,
  "userId": "1"
}
```

orders

```
JSON{
  "items": [
    {
      "title": "iPhone",
      "price": 1000,
      "quantity": 2
    }
  ]
}
```

```
}  
],  
  
"user":{  
  "_id":"1",  
  "name":"Ahmed"  
}  
}
```

Backend كده أنت خلصت المشروع كله من ناحية الـ



أنت فهمت دلوقتي:

- Architecture الـ MVC.
- Express Router.
- Middleware.
- Request / Response Cycle.
- دورها Controllers.
- Models و OOP.
- Classes و this.
- Static Methods.
- Promises و .then/.catch.
- MongoDB CRUD.
- insertOne.
- find.
- findOne.
- updateOne.
- deleteOne.

- \$set.
- \$in.
- ObjectId.
- map.
- find.
- findIndex.
- filter.
- Spread Operator.
- ربط User بال Cart.
- ربط Cart بال Products.
- تحويل Cart إلى Order.

وبصراحة الجزء الجاي اللي أنصحك تعمله قبل ما تعتبر نفسك فاهم المشروع هو **إنك تعيد كتابة المشروع من الصفر من ذاكرتك**. هتكتشف الأماكن اللي محتاجة مراجعة، وساعتها 🔥. "هتكون بدأت تنتقل من "فاهم الكود" إلى "قادر تكتبه